

Lecture 14

Asynchronous Programming

CS 12 Team 25.2
A.Y. 2025–2026, 2nd Sem
Department of Computer Science
College of Engineering
University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
 - (we're working on getting the other TV fixed...)
- **Recording?**
- Important announcements?

Motivating Example

`14/sync.html (as
index.html)`

`14/sync.py`

Motivating Example

Why does clicking the fetch button make the webpage freeze?

Blocking Operations

Blocking operation: Operation must be completed before next operation is executed

Aside: Thread of Execution

Thread: Sequence of instructions executed one after another

- A program is assigned *one thread by default*
- Normally assigned to one CPU core
- Web browsers normally have one thread for web programs

Aside: Thread of Execution

What will the code below output?

Aside: Thread of Execution

```
import time

def f(id: str):
    for i in range(5):
        print(f'{id}: Iteration {i}')
        time.sleep(1)  # Pauses execution for 1 second

f('A')
f('B')
f('C')
```

Blocking Operations

Blocking operation: Operation must be completed before next operation is executed

- Current thread cannot continue execution until operation is done
- Example: `httpx.get()` blocks execution until response is obtained

Disadvantage of Blocking Operations

With only a single thread, blocking operations must be done sequentially

Disadvantage of Blocking Operations

```
import httpx

ranges = [1, 150, 512]

for r in ranges:
    url = f'http://httpbin.org/range/{r}?duration=2'
    print(f'Fetching {url}...')
    res = httpx.get(f'http://httpbin.org/range/{r}?duration=2')
    print(res.text)

print('Done')
```

Disadvantage of Blocking Operations

- URL 1 request sent
- Request/response in flight (2s)
- URL 1 response received
- URL 2 request sent
- Request/response in flight (2s)
- URL 2 response received
- URL 3 request sent
- Request/response in flight (2s)
- URL 3 response received

Total time: 6 seconds

Disadvantage of Blocking Operations

Optimization: Send next request without waiting for response to previous request

- Requesting data from the internet is done in the background
- Multiple requests can be in progress *at the same time*

Nonblocking Operations

Nonblocking operation: Operation runs in “background” (usually I/O); next line/operation is executed immediately even if operation is not yet done

Goal: Print response as soon as it arrives

Nonblocking Operations

How do we run code to handle the completion of a nonblocking operation?

Coroutines

Coroutine: Function capable of pausing its own execution and having it unpaused

- Allows a single thread to *concurrently weave through instructions of multiple functions*
- `yield` pauses the calling coroutine; execution returns to its caller
- `next` continues a paused coroutine (or starts an initialized coroutine)

Coroutines

```
def f():  
    for i in range(5):  
        print(i)  
        yield          #  
        Pauses execution at  
        this point  
  
def main():  
    try:  
        coro = f()      #  
        Initializes a new  
        coroutine (not  
        started)
```

```
while True:  
  
    print('Continuing  
    coroutine...')  
    next(coro)  
    print('Coroutine  
    was paused')  
  
except  
StopIteration:  
    print('Coroutine  
    is done')  
  
main()
```

Concurrency

Concurrent execution: Execution interspersed within the same time interval

- Regular functions vs. coroutines:

Concurrency

```
def f():  
    for i in range(5):  
        print(i)  
        yield
```

```
def main():  
    try:  
        coro = f()  
        while True:  
            print('Continuing coroutine...')  
            next(coro)  
            print('Coroutine was paused')  
  
    except StopIteration:  
        print('Coroutine is done')
```

```
main()
```



```
import time
```

```
def f(id: str):  
    for i in range(5):  
        print(f'{id}: Iteration {i}')  
        time.sleep(1)
```

```
def main():  
    f('A')  
    f('B')  
    f('C')
```

```
main()
```



Event Loop

Event loop: Infinite loop that manages concurrent execution of coroutines

- `while` loop in `main` as event loop with *task queue* (simplified example):

Event Loop

```
def f(id: str):  
    for i in range(5):  
        print(f'{id}: Iteration {i}')        yield  
  
def main():  
    task_queue = [f('A'), f('B'), f('C')]  
  
    while task_queue:  
        coro = task_queue.pop(0)  
        try:
```

```
        next(coro)  
    except StopIteration:  
        continue
```

```
task_queue.append(coro)
```

```
main()
```

Asynchronous Functions

- `async def`: Defines an asynchronous function
 - Calling an `async` function returns an awaitable that must be awaited to be executed
 - Also called nonblocking function (regular functions are called blocking functions)
 - Annotated as `Awaitable[[], ...]` instead of `Callable[[], ...]`
 - Very common in JavaScript

Asynchronous Functions

- `await` : Pauses execution of calling async function
 - Similar to pausing behavior of `yield`
 - Other async functions may execute during this pause
 - Continues when awaitable finishes execution
 - Can only be used inside an async function; infectious (callers must be async as well)

Asynchronous Functions

Sample syntax:

```
import asyncio

async def f():
    await asyncio.sleep(0) # Explained in next slide
```

Asynchronous Functions

- `asyncio`: Built-in Python library for asynchronous execution (see documentation)
 - `.run(f)`: Runs an async function via a built-in event loop (not an async function)
 - `.sleep(n)`: When awaited, continues after n seconds
 - `.gather(f1, f2, ...)`: When awaited, executes async functions `f1`, `f2`, `...` concurrently and ends only after they all terminate

Asynchronous Functions

```
import asyncio

async def f(id: str):
    for i in range(5):
        print(f'{id}: Iteration {i}')
        await asyncio.sleep(0)

async def main():
    await asyncio.gather(*[f('A'), f('B'), f('C')])
    print('Done')

asyncio.run(main()) # Runs event loop
```

Fetch joke example (async HTTP request version)

`14/async.html (as
index.html)`

`14/async.py`

JSON Format

JSON: JavaScript Object Notation

- Standardized way of encoding data as text on the web

JSON Format

JSON value

Literal

Null

`null`

Boolean

`true` and `false`

Number

`12` and `-12.31`

String

`"CS 12"`

Array of JSON values

`[null, true, 0, "hello"]`

Object: Dictionary of string-
JSON value pairs

`{}, {"key": "value", "a": 1, "b": true}`

Deserialization

- *Serialization*: converting complex data objects into byte/string format
- *Deserialization*: reconstructing complex data objects from a serialized format (e.g. JSON, XML, binary)

Deserializing a JSON string into a dict

- Deserialization of a JSON string via `loads` function of built-in `json` library

```
import json

json_example = '''
{
    "names": ["Beatrice", "Michel", "Mamiya"],
    "count": 3
}
'''
```

```
data = json.loads(json_example) # Returns dict equivalent  
                                   # of JSON in input string  
  
print(data['names'][1])          # Michel
```

Deserializing a JSON string into a dict

- Note 1: JSON `null` becomes Python `None`
- Note 2: Raises `json.decoder.JSONDecodeError` if string is not valid JSON

Serializing a dict as a JSON string

- Serialization of Python values into JSON string via `dumps` function of built-in `json` library

```
import json

data = {
    "names": ["Beatrice", "Michel", "Mamiya"],
    "count": 3
}

json_str = json.dumps(data)           # Turns input value into JSON
```

```
# and returns it as a string
```

```
print(json.loads(json_str) == data) # True
```

Serializing a dict as a JSON string

- Note: Raises `TypeError` if value cannot be serialized into JSON

Application Programming Interface (API)

- *Application Programming Interface (API)*: Defined means of interaction with program or usage of code
- Sample PokeAPI endpoint: <https://pokeapi.co/api/v2/pokemon/> (see [documentation](#))

Application Programming Interface (API)

14/json_api.py

HTTPX

- *HTTPX*: third-party library containing fully featured HTTP client for Python 3, which provides sync and async APIs
- `pip install httpx`
- Documentation: <https://www.python-httpx.org>
- Async guide: <https://www.python-httpx.org/async/>
 - Supports built-in `asyncio` as an async environment

Demo: PyScript mini webapp showing ten random Pokemon